



N O D E F L O W

HYBRID EDGE / CLOUD INFERENCE ORCHESTRATOR FOR KNOWLEDGE GRAPHS

PROJECT DOSSIER · ASSEMBLYAI VOICE AGENT HACKATHON · SEPTEMBER 2026

The canvas you can talk to — and the inference that stays honest.

A native desktop app that turns scattered ideas into a structured, human-approved knowledge graph. You speak; the transcription becomes a **plan of operations** **validated against the real graph**, shown for approval before a single node moves. Behind it, a router decides per task whether inference runs locally at zero marginal cost or in the cloud at a price it accounts for and can justify.



TRY IT nodeflow-demo.vercel.app · no install, real voice path

SOURCE github.com/NeonEden/NodeFlow · MIT

AUTHOR TOMAS.WAV · Tomas Pieruz · 16 September 2026 · v0.3.5

Tauri v2

Rust · axum

React 19

AssemblyAI v3

Kokoro TTS

Azure AI Foundry

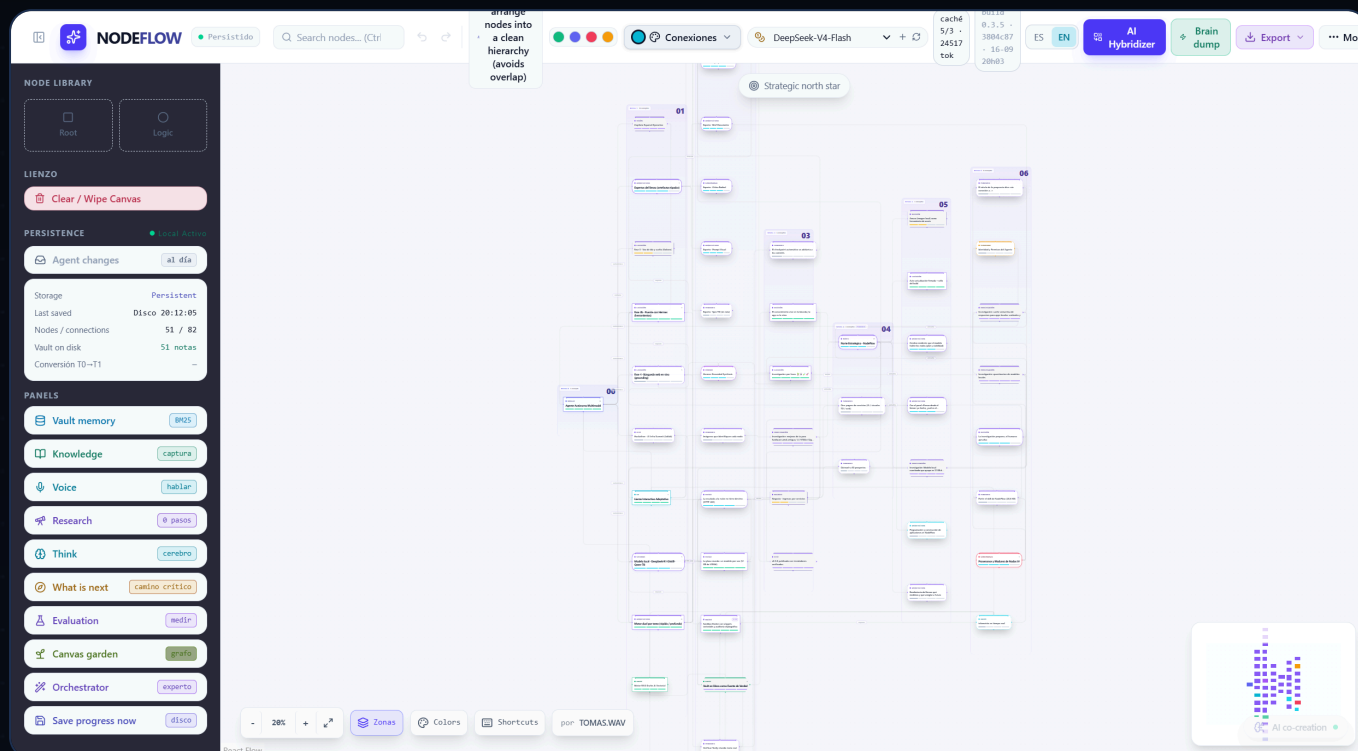
Ollama

Obsidian vault

ES / EN

01 What it is, in one page

NodeFlow is a desktop app and a local server. The app is a canvas where knowledge lives as **nodes and typed edges**; the server is the thing that thinks, decides where to run each inference, prices it, caches it, and never writes to the graph without a human approving it.



The real app: knowledge graph with maturity rings, typed edges, category lenses and zones. Interface shown in English; the same UI runs in Spanish from a typed catalogue.

The three things it fixes

- **Waste:** operations that never needed a frontier model still get sent to one.
- **Opacity:** nobody can say what an artifact cost in tokens or dollars.
- **Amnesia:** the same prompt over the same content is recomputed forever.

What makes it infrastructure

- The motor is **data, not code**: the catalogue is built from what actually exists.
- The **code validates** every structured field against the live graph vocabulary.
- The **knowledge outlives the tool**: Markdown + YAML in your own Obsidian vault.

One sentence for the judges: NodeFlow is the part of the AI stack that decides *where* each inference should run, *what* it costs, and *whether* a human has approved the result — and it proves all three with numbers read off the running app.

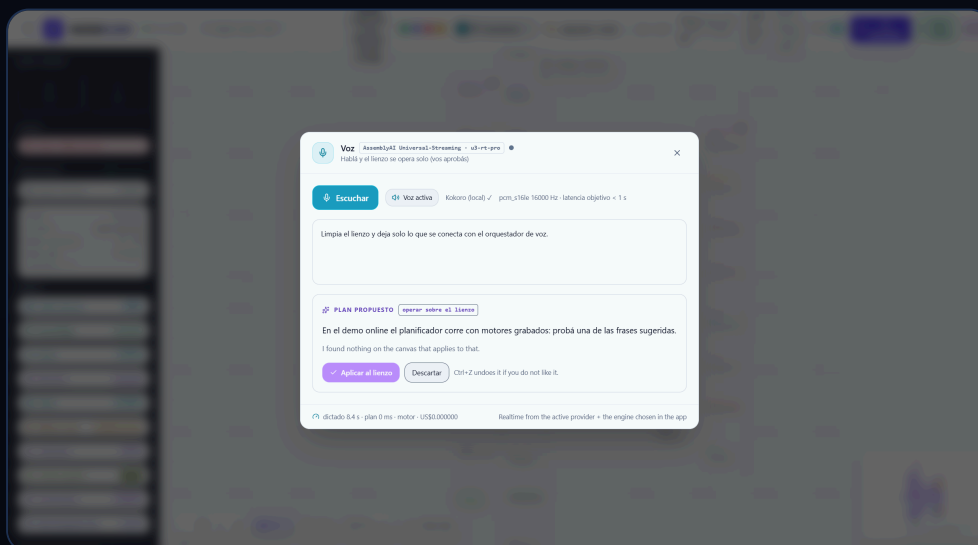
Who it is for

Teams whose notes, specs or research are private by nature (product, research, agencies under NDA, legal / health / finance knowledge work) and any shop paying per token for work a local 2B model can already do.

Speaking is not a gimmick bolted onto a canvas. The spoken sentence has to become a **typed, validated plan** over a graph the engine can actually see — and the human still approves it. Everything below was verified against the real WebSocket, not in a mock.

The chain, in order

STEP	WHAT HAPPENS	WHERE IT IS VERIFIED
1 · ear	Microphone → PCM 16-bit / 16 kHz → binary frames over <code>wss://streaming.assemblyai.com/v3/ws</code> . Partial and final frames arrive live.	AssemblyAI Universal-Streaming v3 · model <code>u3-rt-pro</code> declared explicitly
2 · token	The browser never sees an API key: it asks our backend for a short-lived token (<code>/api/voz/jwt</code>) and the WebSocket is opened with that.	key lives in the OS keyring; the front end only carries a token
3 · plan	The transcript is interpreted as operations over the real graph: create, link, focus, collapse, question, or delegate what the canvas cannot answer.	live engine run: 3,555 ms · 3,582 tokens
4 · guard	Server-side validation against ids that exist, only allowed actions, hard caps. A plan that invents an id dies here.	validator in Rust, per-command, with tests
5 · approve	Nothing moves until a human clicks. Every applied change keeps its audit trail.	the proposal queue, visible in the app
6 · answer	The assistant speaks only when speaking adds something: structural moves and discarded proposals are spoken; creating a node stays silent because you can see it.	Kokoro TTS locally · rule in code with tests



Real capture: the panel during a dictation. The engine and its model are declared by the backend, the transcription is the real one, and the plan card appears only when one exists. When the planner is unreachable, the public demo answers with a recorded plan instead of an error — the guardrail, visible.

The plan the engine returned for that dictation

Spoken: "Limpiá el lienzo y dejá solo los nodos que se conectan con el orquestador de voz."

```
{
  "intencion": "comando",
  "respuesta": "Voy a filtrar el lienzo dejando
    solo los nodos que se conectan con el
    orquestador de voz.",
  "hablar": true,
  "comandos": [
    { "accion": "enfocar",
      "criterio": "conexión con orquestador de voz",
      "nodos": [ "n-ag-fase-3-voz-de-ida-y-vuelta-kokoro",
        "n-ag-fase-3b-puente-con-hermes-herramientas",
        "n-ag-voz-con-assemblyai-en-vivo-...",
        "n-ag-cerebro-residente-...",
        "node-voz-1789344440460-0",
        "node-hybrid-1789579118521",
        "node-voz-1789579916304-0" ] }
  ]
}
```

source: openai@DeepSeek-V4-Flash · modo auto · 3.555 ms · 3.582 tokens (3.360 prompt + 222 completion)
· cache miss

Why the ids matter. The plan did not describe nodes: it named seven that exist, including the voice node created the same day. That is only possible because the engine receives the real graph as context and the code checks every id before the plan is offered.

The language travels with the voice

The interface language is not a coat of paint: switching to English changes the transcription language and the voice you hear (a native Spanish or English voice, not an accent). Transcription and answer stay in the same language as the person using the tool, and the catalogue makes a missing translation a build error.

The measured loop, end to end: 4.1 s of Spanish speech → live transcript → plan in 3.555 ms with 3.582 tokens → server-side validation → human approval. Nothing in that chain is narrated; every number came out of the running app.

<https://nodeflow-demo.vercel.app>

The same frontend, served from a serverless function, against a simulated API built from **24 real responses harvested from the running backend**. No account, no install, and the voice path is real: the browser talks to AssemblyAI with a token minted server-side.

What is real in the demo

- The complete UI: canvas, panels, gardens, editors, the ES/EN switch.
- The **voice**: real WebSocket to AssemblyAI, real transcription.
- The interaction model: nothing applies without approval.

What is simulated (and declared)

- The graph is a fixture: 51 nodes / 82 edges, sanitized before publishing.
- The planner runs live when a motor is wired, and otherwise answers with a recorded plan.
- Nothing in the demo touches a vault or a local model.

The guardrails that make a demo survive a jury

RISK	WHAT THE DEMO DOES
Model cost / abuse	700-token cap per answer • 20 s timeout • rate limit 8 requests/minute per IP
The key leaking to the browser	key is server-side only; a static deploy once served local files – that hole was closed and verified with a 404 from outside
"The AI broke" in front of a judge	silent fallback to recorded plans, flagged in the answer so it is never a lie
Confusing output with data	if the model echoes the schema instead of the answer, the code notices and retries once

Honest note for the record. The planner inside the public demo prefers a recorded plan when the live motor is not wired for that environment; the screenshot in section 02 shows exactly that state, and the panel says so instead of pretending. The live numbers quoted in this dossier come from the desktop app — the real product — measured on this machine.

What a visitor can do in three minutes

- Open the canvas and read a real knowledge graph, with maturity and lineage visible.
- See a plan, and see that nothing is applied until it is approved.

Layers

LAYER	RESPONSIBILITY	TECHNOLOGY
Shell	Native window, updater, OS integration, keyring access.	Tauri v2 (Rust) · WebView2
Front end	Canvas, panels, lenses, zones, i18n catalogue, approval UI.	React 19 · React Flow · Tailwind
Server	58 HTTP routes on <code>127.0.0.1:37371</code> ; graph, AI, voice, vault, agent, keys.	Rust · axum · tokio
Router	Per-task decision local vs cloud; rate table; two-tier cache.	costo.rs · semantica.rs
Edge	Drafts, local reasoning, image reading. No quota, works offline.	Ollama · granite3.3:2b · deepseek-r1:7b · qwen2.5vl:7b
Cloud	Escalation for depth, with declared motors and declared rates.	Azure AI Foundry · DeepSeek · Gemini
Speech	Ears and voice, swappable without the UI knowing.	AssemblyAI v3 · Speechmatics · Kokoro TTS

Design decisions worth defending

- **Tauri over Electron:** 37.7 MB resident instead of hundreds, GPU left for inference.
- **Own i18n catalogue over react-i18next:** zero dependencies, and a missing translation breaks the build instead of shipping a half-translated screen.
- **Vault as source of truth:** the app can be replaced; the customer's knowledge cannot be taken hostage.
- **No API-management layer in front:** an AI gateway was provisioned, measured, and deleted — a hop that authenticated nothing and burned ~USD 150/month in credit on zero calls.

Security model, in four lines

- Keys live in the **OS keyring**; config files keep field names, never values.
- An endpoint reports where each key lives, never the key.
- The agent proposes; only explicit human approval writes.
- Whatever is deployed is checked with a request from outside, not with a success message.

05

Measured numbers

Every figure below was measured on this machine or read off the running app — none is estimated.

METRIC	VALUE	HOW IT WAS MEASURED
Resident memory	37.7 MB RSS	Get-Process app, v0.3.5 release build
Knowledge graph in daily use	57 nodes · 89 edges	GET /api/graph/state · 57 notes on disk agree
Inference avoided by the cache	32,382 tokens	14,031 exact + 18,351 semantic · 112 entries · cap 300
Raw idea → approved AI artifact	10 conversions · 22.8 min avg	GET /api/metrics, timestamps written by the app itself
Live plan from one dictation	3,555 ms · 3,582 tokens	POST /api/ai/action, DeepSeek-V4-Flash, cache miss
Engine planilla, same 5 real tasks	5/5 · 4/5 · 2/5	POST /api/ai/evaluar — see next section
Rust unit tests	216 passed / 0 failed	cargo test --lib on the shipped tree
TypeScript build	0 errors	tsc --noEmit
Own source lines	~40,600	19.0k TS/TSX · 19.5k Rust · 2.1k scripts
HTTP routes	58	registered in the axum router
Installer size	4.4 MB NSIS · 6.8 MB MSI	release bundle, signed updater artifacts
Local speech synthesis	2.4× faster than real time	Kokoro 82M parameters, ~340 MB, no torch

57 / 89

GRAPH IN USE
notes and nodes agree

32.4k

TOKENS NEVER RECOMPUTED
two-tier cache

0.6 min

BEST RAW IDEA → ARTIFACT
median session 22.8 min

Why the T0→T1 metric exists. The app times its own value: how long between the raw brain dump and the first AI artifact a human approved. It is the one number that says whether the tool is actually helping, and it is the number a buyer would ask for.

The same five real canvas tasks were run through every candidate motor, with the result checked in code. The routing promoted whatever won — which is how a fast cloud model ended up ahead of a reasoning model.

MOTOR	SCORE	AVERAGE LATENCY	READING
azure · DeepSeek-V4-Flash	5 / 5	5.5 s	Wins the canvas loop. Depth where it matters, without making the user wait.
azure · grok-4.6	4 / 5	95 s	Correct answers at a latency nobody tolerates in a live canvas. Registered, not promoted.
local · granite3.3:2b	2 / 5	5.7 s	Keeps the tasks where latency is everything; not enough for the hard five.
azure · gpt-5-mini	—	—	Rejects any <code>temperature</code> but the default (HTTP 400). Recorded as a documented limit instead of hidden.

How a call is priced

Rates come from configuration, and **a model without a declared rate reports “unknown cost”** — never a fake \$0. Every generated artifact carries its provider, model, milliseconds, tokens and cost, and the two-tier cache adds what was avoided.

The two cache tiers

- **Exact:** node + prompt + provider + schema + a versioned contract constant.
- **Semantic:** cosine over embeddings ≥ 0.92 , degrading to a dependency-free token comparison when there is no key or no network.

The mistake that made this section possible. The first cache implementation hashed a prompt containing the current timestamp: identical runs scored 6 misses and 0 hits. The key became a versioned contract with tests — and now the same cache saves 32,382 tokens of inference.

1 · The model proposes, the code validates against the live vocabulary of the graph

A JSON grammar guarantees the shape of an answer, never its truth: the local 2B once returned a maturity of `100` in perfectly valid JSON. Every structured field is checked against what the graph actually contains — and now the same rule protects the spoken plans: only ids that exist, only allowed actions, hard caps.

2 · A cache key that cannot drift

A versioned contract plus a semantic tier with a graceful degradation path when there is no key or no network. This is the mechanism that pays for the whole router: 32,382 tokens of inference avoided.

3 · Every action is a proposal with an audit trail

The agent — local or remote, typed or spoken — never writes to the graph. It proposes; the change is validated; it lands after explicit human approval. That is the difference between an assistant and an autonomous writer, and it is what makes the spoken loop safe enough to use in front of an audience.

4 · Lossless condensation with lineage

A selection collapses into one macro-node that keeps its nodes and edges inside (`collapsed_nodes_count` , `lineage_node_ids`). Double-click restores the original sub-graph. Pruning that cannot lose anything.

5 · Routing decided by a measured planilla, not by reputation

Five real tasks through every candidate motor, results verified in code, winners promoted. It is why the product can say why it chose a motor — and why it can refuse one that is technically stronger.

The through-line: four of the five mechanisms exist to make an AI system auditable — what ran, where, at what price, with whose approval. None of them is a prompt.

A dossier that only lists strengths is a brochure. This is what is not built, in the order I would build it.

NOT BUILT YET	WHY IT MATTERS	WORKAROUND TODAY
Streaming (SSE) into the active node	Today a node appears when the whole JSON phase closes; local runs show 6–8 s of narrated wait.	the wait is narrated, not silent
Per-task Local/Cloud switch on the canvas	The router already accepts a per-task override; the control is not surfaced next to the node.	global selector in the top bar
Remaining interface translation	85 keys cover sidebar, HUD, voice and agent panels; 339 strings exist in total.	the app is fully usable in Spanish
macOS / Linux builds	The product is Windows-only today.	the core is Rust and portable
OS code-signed installers	The updater signature and channel work since v0.3.1; the Authenticode certificate is missing.	signed updater artifacts, manual publish

The path, in three moves

Now

The app is published with a working updater, and the public demo lets a prospect try the interaction without installing anything.

Next

Service work built on it — agent automation, private-knowledge workflows — with the app as the lead magnet.

Later

A paid tier around team sync, the approval audit trail and per-artifact cost governance.

What the product refuses to do. It will not claim a capability it has not measured, it will not hide a motor that fails through a given path, and it will not apply a change nobody approved. Those three refusals are the feature.

ITEM	VALUE
Hackathon	AssemblyAI Voice Agent Hackathon — lablab.ai · 1–30 September 2026
Project	NodeFlow — hybrid edge/cloud inference orchestrator for knowledge graphs
Repository	github.com/NeonEden/NodeFlow · MIT · public
Live demo	nodeflow-demo.vercel.app
Voice stack	AssemblyAI Universal-Streaming v3 (u3-rt-pro), PCM 16 kHz, token minted server-side) + Kokoro TTS locally
What is submitted	The desktop installer, the web demo, the demo video, the slide deck, and this dossier
Author	TOMAS.WAV · Tomas Pieruz · Argentina (UTC-3)

Why this is an agent, not a transcription app

- The transcript is not displayed and forgotten: it becomes **operations on real state**.
- The agent has tools: it creates, links, focuses, condenses, questions, and delegates when the canvas cannot answer.
- It decides **when to speak** in code, with tests — not by vibes.
- It is **governed**: server-side validation, approval, audit trail, and a cost trail for every call it makes.

Try in 60 seconds

1. Open nodeflow-demo.vercel.app.
2. Press the **Voice** panel in the sidebar.
3. Switch to Spanish or English — the voice follows.
4. Press **Listen**, speak one instruction, and read the plan.
5. Note that nothing moved until you approve it.

Run it locally

```
npm install
ollama pull granite3.3:2b      # optional:
local drafts
npx tauri dev                  # app +
Rust backend :37371
cargo test --manifest-path src-
tauri/Cargo.toml --lib
npm run demo:build && npm run demo # web
demo :4173
```

Requirements: Windows 10/11 x64, WebView2, Node 22+, Rust 1.77+.

10 Example email (ready to send)

A real pending item, written the way it should be sent: specific, with the request ids that support already returned, and without asking for anything that has not been measured first.

To support@assemblyai.com

Subject LLM Gateway access – account with working Streaming STT (request ids inside)

Hi team,

I'm building NodeFlow (github.com/NeonEden/NodeFlow), a desktop knowledge-graph tool where you speak and the canvas executes a validated plan. The voice path is **already running on Universal-Streaming v3** in production — short-lived tokens minted by our own backend, PCM 16 kHz over the v3 WebSocket, and Spanish verified end to end against the live socket.

I would like to move the reasoning step onto your **LLM Gateway** so that speech and reasoning share one key, one bill and one data-residency story. Right now every model I try through the gateway answers:

```
HTTP 400 · "Your account does not have access to this LLM Gateway model"
```

`GET /v1/models` returns 200 and lists the catalogue, so the key itself works — the models are what appear to be gated. I tried: `gemini-3.5-flash-lite`, `gemini-3.6-flash`, `gpt-oss-20b`, `claude-haiku-4-5`, `minimax-m3`.

Could you enable LLM Gateway access for the account, or tell me what I need to complete first? I have the two support request ids from the in-app attempts: `f44b9ba6...` and `b1308b8e...` (full ids in the repository under `docs/hackathon/pedido-llm-gateway.md`).

Happy to share a minimal reproduction — it is a five-line `curl` against `/v1/chat/completions` that returns the 400 above.

Thanks,

Tomas Pieruz · TOMAS.WAV

NodeFlow – the canvas you can talk to

What makes this email work as an example: it states what already works before asking for anything, it quotes the exact error, it lists the attempts, it gives the support ids, and it offers a reproduction instead of a complaint.

A Appendix · API surface (58 routes)

Everything the desktop app does, another program can do. All routes are local by default.

Graph

/api/graph/state · node · node/delete · edge · merge · summary · garden · garden/fix · tidy · prune · siguiente

AI & routing

/api/ai/action · motores · motor · proveedor · cache · evaluar · delegar · investigar

Voice

/api/voz/estado · jwt · proveedores · proveedor · decir · dialogo

Knowledge & vault

/api/knowledge/capture · draft · preview · /api/vault/info · search · note · memory · reindex

Agent (human in the loop)

/api/agent/pending · approve · reject

Calibration & keys

/api/hitl/feedback · preferences · profile · recalibrate · reset · /api/claves/estado · migrar

The rest of the dossier lives in the repo

README.md	the product page, with the same measured numbers
docs/BENCHMARKS.md	hardware, local model throughput, the planilla and the cache effect
docs/EVALUACION.md	the engine planilla task by task
docs/adr/	why Tauri, why a hash cache, why code-side validation, why hybrid routing
docs/hackathon/submission.md	the submission pack and the video script
docs/DEVLOG.md	the development log, generated from the commit history

Contact. Tomas Pieruz — TOMAS.WAV · Argentina (UTC-3) · repository [NeonEden/NodeFlow](#) · demo [nodeflow-demo.vercel.app](#)